

Identificación de las clases en la especificación de requerimientos

Daniela Patricia Feversani y David Roldán

8 de octubre de 2025

1. Introducción

Ahora empezaremos a diseñar el sistema de gestión de citas que presentamos en la Práctica 2. En esta práctica identificaremos las clases necesarias para crear el sistema planteado, analizando los sustantivos y las frases nominales que aparecen en la especificación de requerimientos. Presentaremos los diagramas de clases de UML para modelar las relaciones entre estas clases. Este primer paso es fundamental para definir la estructura de nuestro sistema.

2. Visión general del sistema

Para comenzar nuestro proceso de Diseño Orientado a Objetos (DOO), vamos a identificar las clases requeridas para crear el sistema de gestión de citas. Más adelante describiremos estas clases mediante el uso de los diagramas de clases de UML y las implementaremos en C++. Primero debemos revisar la especificación de requerimientos que ya hicimos en la Práctica 2 e identificar los sustantivos y las frases nominales clave que nos ayuden a identificar las clases que conformarán el sistema. Tal vez decidamos que algunos sustantivos y frases nominales sean atributos de otras clases en el sistema. Tal vez concluyamos que algunos de los sustantivos son irrelevantes para nuestro sistema y que por lo tanto no deben modelarse. A medida que avancemos en el proceso de diseño iremos descubriendo nuevas características del sistema que deberemos o no modelar.

Vamos a crear clases sólo para los sustantivos y frases nominales que tengan importancia en el sistema. Por ejemplo, no modelamos el hospital como clase, ya que el hospital no es una parte del sistema. El hospital solo quiero que nosotros construyamos el sistema de gestión de citas. En nuestro sistema, que incluye la creación de citas, modelaremos las distintas acciones que pueden ser realizadas: Crear cita, confirmar cita, finalizar cita, cancelar cita. Por lo tanto, parece evidente que necesitaremos una clase que contenga los atributos y los métodos necesarios para una cita. Por ejemplo, para ver una cita necesitamos la fecha, la hora de la cita y la especialidad seleccionada. Sin embargo, no necesitamos datos adicionales a los que ya se encuentran en la propia cita. Lo que es más, las distintas

clases correspondientes a las distintas acciones que se pueden realizar en el sistema muestran comportamientos únicos. Para ver una cita se requiere mostrar la fecha, la hora y la especialidad, mientras que para recoger un Ticket se necesita el identificador del paciente y se devolverá un ticket con la cita correspondiente.

Una vez se han encontrado las clases en la Práctica 2, ya podemos modelar las clases en nuestro sistema. En el proceso de diseño escribimos los nombres de las clases con la primera letra en mayúscula (una convención en UML), como lo haremos cuando escribamos el código de C++ para implementar nuestro diseño. Si el nombre de una clase contiene más de una palabra, juntaremos todas las palabras y escribiremos la primera letra de cada una de ellas en mayúscula. Esto se conoce como notación camello (CamelCase). Utilizando esta convención, vamos a crear las clases. Construiremos nuestro sistema mediante el uso de todas estas clases como bloques de construcción. Sin embargo, antes de construir el sistema, debemos comprender mejor la forma en que las clases se relacionan entre sí.

3. Modelado de las clases

UML nos permite modelar, a través de los diagramas de clases, las clases en el sistema ATM y sus interrelaciones. La Figura 1 representa la clase Cita. En UML, cada clase se modela como un rectángulo con tres compartimentos. El compartimento superior contiene el nombre de la clase, centrado horizontalmente y en negrita. El compartimento intermedio contiene los atributos de la clase. El compartimento inferior contiene los métodos de la clase.

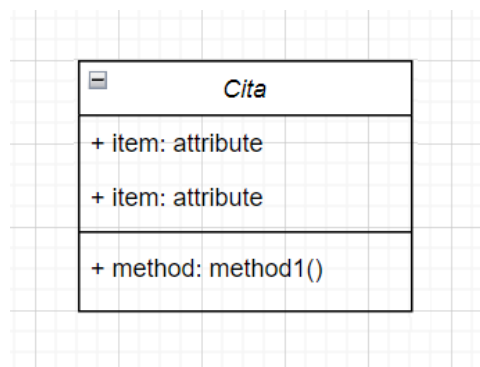


Figura 1: Representación de una clase en UML

Los diagramas de clases también muestran las relaciones entre clases del sistema. En la Figura 2 se muestra como nuestras clases Cita y Paciente se relacionan la una con la otra. Observa que alguno de los rectángulos no están subdivididos en compartimentos. UML permite suprimir los atributos de las clases y sus operadores de esta forma, cuando sea apropiado, para crear diagramas más legibles. Un diagrama de este tipo se denomina diagrama con elementos omitidos: su información, tal como el contenido de los compartimentos segundo y tercero, no se modela. Más adelante colocaremos información en estos compartimentos.

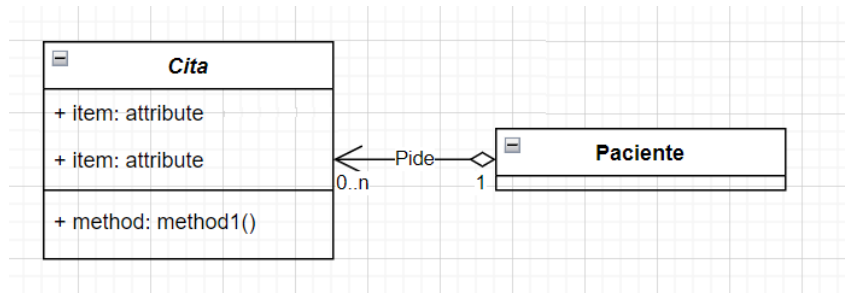


Figura 2: Diagrama que muestra una asociación entre clases

Actividad: Modela el conjunto de las clases del sistema que habías determinado. Para ello usaremos la herramienta Drawio (Figura 3), aunque si conoces otra herramienta no hay ningún problema en que la utilices. Es una herramienta muy simple, básica, pero a la vez muy potente. Todos los elementos disponibles están a mano izquierda. Para usar cualquiera de ellos basta con hacer doble click sobre él o arrastrarlo.

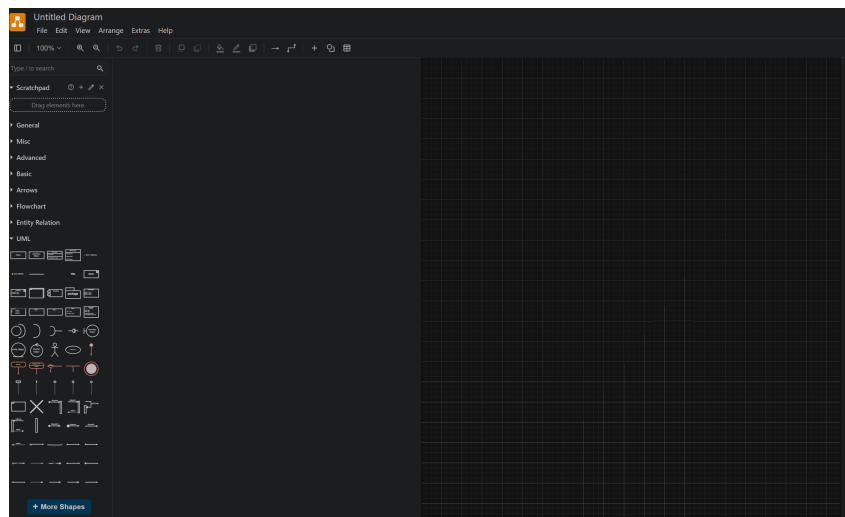


Figura 3: Interfaz de la herramienta Drawio

En la Figura 2, la línea sólida que conecta a las dos clases representa una **asociación**, una relación entre clases. Los números cerca de cada extremo de la línea son valores de multiplicidad, que indican cuántos objetos de cada clase participan en la asociación. En este caso, al seguir la línea de un extremo al otro se revela que, en un momento dado, un objeto Cita participa siempre con un objeto paciente, ya que una cita siempre tiene que estar asociada a un paciente que la ha solicitado. UML puede modelar muchos tipos de multiplicidad. En la Tabla 1 se listan y explican los tipos de multiplicidad.

Una asociación puede tener nombre. Por ejemplo, la palabra **Pide** en la línea de la asociación que conecta a las clases **Paciente** y **Cita** en la Figura 2 indica el nombre de la asociación. Esta parte del diagrama se lee así: *Un objeto de la clase Paciente pide cero o varios objetos de la clase Cita*. Los nombres de las asociaciones son direccionales, como lo indica la punta de la flecha rellena. Por lo tanto, sería inapropiado, por ejemplo, leer la

Símbolo	Significado
0	Ninguno
1	Uno
m	Un valor entero
0..1	Cero o uno
m, n	m o n
$m..n$	Cuando menos m , pero no más que n
*	Cualquier entero no negativo (cero o más)
0..*	Cero o más (idéntico a *)
1..*	Uno o más

Tabla 1: Tipos de multiplicidad

anterior asociación de izquierda a derecha como *Un objeto de la clase Cita pide un objeto de la clase Paciente*. Cabe destacar que un objeto de una clase puede tener varias asociaciones diferentes. Es más, un objeto de una clase puede tener varias asociaciones con un objeto de una clase. A estas diferentes formas de participación se le conocen como **Roles**. Por ejemplo, siguiendo el ejemplo anterior, una Cita a su vez está relacionada con un médico. Además, una clase puede estar compuesta por otras clases mediante **relaciones de composición**. La Figura 4 muestra un ejemplo de como el sistema de gestión de citas está compuesto por uno o varios robots asistentes.

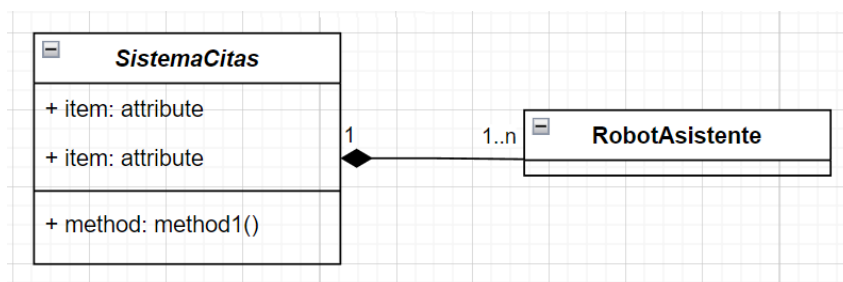


Figura 4: Ejemplo relación de composición

La composición implica una relación en todo/en parte. La clase que tiene el símbolo de composición (el rombo sólido) en su extremo de la línea de asociación es el todo (en este caso, SistemaCitas) y las clases en el otro extremo de las líneas de asociación son las partes. En este caso, RobotAsistente. La composición de la Figura 4 indican que un objeto de la clase SistemaCitas está compuesto por entre 1 y n RobotAsistente. De acuerdo con la especificación de UML, las relaciones de composición tienen las siguientes propiedades.

- Solo una clase en la relación puede representar el todo. Es decir, el rombo puede colocarse sólo en un extremo de la línea de asociación.
- Las partes en la relación de composición existen solo mientras exista el todo, y el todo es responsable de la creación y destrucción de sus partes. Por ejemplo, el acto de construir un sistema incluye la manufactura de sus partes. Lo que es más, si el sistema se destruye, también se destruirían los RobotsAsistentes.

- Una parte puede pertenecer sólo a un todo a la vez, aunque esa parte puede quitarse y unirse a otro todo, el cual entonces asumirá la responsabilidad de esa parte.

Los rombos sólidos en nuestros diagramas de clases indican las relaciones de composición que cumplen con estas tres propiedades. Si una relación *tiene un* no satisface uno o más de estos criterios, UML especifica que se deben adjuntar rombos sin relleno a los extremos de las líneas de asociación para indicar una agregación, que es una forma más débil de composición. Por ejemplo, podríamos querer definir que nuestro SistemaCitas tuviera o no RobotsAsistentes, o que los RobotsAsistentes de nuestro SistemaCitas fueran conectados a otro SistemaCitas.

4. Ejercicio

Llegados a este punto, ya hemos identificado las clases en nuestro sistema de gestión de citas, aunque tal vez descubramos otras a medida que avancemos con el diseño y la implementación. Como ejercicio final, completa el diagrama de clases para el sistema planteado. Este diagrama debe modelar las clases que ya habías modelado en la Actividad propuesta así como las asociaciones entre ellas que podemos inferir de la especificación de requerimientos. Exporta el resultado final a formato png. Deberás subir al Aula Virtual los ficheros practica3.png y practica3.drawio..